Exercise Sheet 07

The Armoury

House rules.

- One file per exercise, in `my_work/`. Exercise 1 is done in the terminal, no file.
- Run everything **from the lesson folder**, with the virtual environment active (your prompt starts with `(.venv)`).
- Type the **Given** lines first, then write the rest. Compare with the **expected output**.
- Hand in: commit and push to `lesson-07`, open a pull request. Check `git status` first: `.venv/` must NOT be in it.

Exercise 1 An armoury of your own

10 min · terminal only

> **Raven from Orrin of the Clockworks:** *"Before you install anything: a virtual environment. It's a private armoury for this project, so whatever you forge here can't break anything else in the guild. Also I don't have to hear about it."*

IN THE TERMINAL, INSIDE THE LESSON FOLDER

```
python -m venv .venv            # build the armoury (once per project)
.venv\Scripts\activate          # open it: the prompt now starts with (.venv)
pip install -r requirements.txt # install what's on the list
pip list                        # what's in the armoury?
python -c "import rich; print('rich is ready')"
```

(a) Run the five commands. `pip list` shows `rich` and a few friends it brought along. Version numbers will differ from your neighbour's; that's fine.

(b) Open `requirements.txt` in VS Code and read it. That one line is the whole shopping list.

(c) Run `git status`. If `.venv/` shows up as untracked, tell Lars: the `.gitignore` should hide it.

EXPECTED OUTPUT (LAST COMMAND)

```
rich is ready
```

Hint: "Running scripts is disabled"? Run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once, then activate again. New terminal = activate again. VS Code usually does it for you once it has noticed the `.venv` folder.

Exercise 2 Borrow from the standard library

8 min · `ex2_maths.py`

> **Decree from Queen Aldara:** *"Captain Siv says I may no longer say 'the average reward' without saying which average. Also, if I split eight heroes into parties of three, how many parties is that? Orrin laughed."*

GIVEN (EIGHT REWARDS FROM THE NOTICE BOARD, IN CROWNS)

```
import math
import statistics

rewards = [260, 550, 620, 670, 310, 1180, 430, 2710]
```

(a) Print the mean (2 decimals), the median, the standard deviation (`statistics.stdev`, 2 decimals) and the range (max minus min).

(b) Print how many parties of 3 you need for 8 heroes (`math.ceil`) and how many *full* parties you get (`math.floor`).

(c) Print the board's total rounded *up* to the nearest 1,000 crowns, and the square root of 1019 to 3 decimals.

EXPECTED OUTPUT

```
Mean reward:   841.25
Median reward: 585.0
Std dev:       807.05
Range:         2450
Parties of 3 for 8 heroes: 3
Full parties of 3: 2
Board total, rounded up to the nearest 1,000 cr: 7000
Square root of 1019: 31.922
```

Hint: Nobody memorises the standard library. Type `python`, then `import statistics`, then `help(statistics.stdev)`. `q` gets you out of the help page. "Round up to the nearest 1,000" is `ceil(x / 1000) * 1000`. And look at mean vs median: one Wyrm drags the mean up.

Exercise 3 Dates without tears

12 min · `ex3_dates.py`

> **Raven from Pell, the Quartermistress:** *"Years in the guild, for the long-service runes, as of today, 5 April. In Excel this is a DATEDIF and a prayer. And muster day is the 25th. I know when muster day is. I want the script to know."*

GIVEN

```
from datetime import date, timedelta

today = date(2026, 4, 5)
joined = {"Wren": "2014-06-21", "Bram": "2019-11-04", "Tamsin": "2025-02-28"}
```

(a) For each hero: turn the text into a date with `date.fromisoformat()`, work out the days since then (`(today - start).days`) and the years (divide by 365.25). Print the name in 8, the date as "21 Jun 2014" and the weekday in brackets, then the years with 1 decimal.

(b) Muster day is the 25th. Build this month's muster; if it has already gone, use next month's. Print it in ISO format and how many days away it is.

(c) The Wyrm contract must be done within 90 days from today. Print the deadline as dd/mm/yyyy.

EXPECTED OUTPUT

```
Wren     joined 21 Jun 2014 (Saturday), 11.8 years ago
Bram     joined 04 Nov 2019 (Monday), 6.4 years ago
Tamsin   joined 28 Feb 2025 (Friday), 1.1 years ago
Next muster: 2026-04-25 in 20 days
Wyrm contract deadline: 04/07/2026
```

Hint: Dates format inside f-strings: `f"{start:%d %b %Y}"`, `f"{start:%A}"`. Codes: `%d` day, `%m` month number, `%b` short month name, `%Y` year, `%A` weekday. `today + timedelta(days=90)` does the calendar maths for you.

Exercise 4 Your own module

12 min · `questkit.py` + `ex4_use_questkit.py`

> **Raven from Orrin:** *"You've now typed `paid / (party_size + 1)` into six different files. The seventh time, forge it into a module and import it. Future Jen will send a thank-you raven."*

GIVEN (IN `MY_WORK/QUESTKIT.PY`)

```
"""Quest arithmetic for the Adventurers' Guild of Thornmere. Import me, don't copy me."""
```

(a) In `questkit.py`, write four functions with docstrings: `share(paid, party_size)` (one hero's loot share: the leader takes a double share, so the loot is cut into `party_size + 1` shares; 2 decimals), `per_day(paid, days)` (2 decimals), `fled_payout(reward, rate=0.25)` (what the guild pays a party that fled; 2 decimals) and `crowns(amount)` (commas, 2 decimals, `" cr"` on the end). No `print` anywhere in this file.

(b) In `ex4_use_questkit.py`: `import questkit`, then for the three quests below print the quest id, one share (as crowns, right-aligned in 12) and the crowns per day. Then what the guild pays for fleeing the Wyrm (2710 cr reward) at the usual rate, and at a third.

```
quests = [("Q0002", 550, 4, 7), ("Q0011", 1380, 5, 10), ("Q0042", 2710, 6, 14)]
#          id       paid party days
```

EXPECTED OUTPUT

```
Q0002     110.00 cr a share, 78.57 cr a day
Q0011     230.00 cr a share, 138.00 cr a day
Q0042     387.14 cr a share, 193.57 cr a day
Fled from the Wyrm: 677.50 cr
Fled, if Siv pays a third: 903.33 cr
```

Hint: Both files must be in `my_work/`, and you run `python my_work/ex4_use_questkit.py`: Python looks for modules next to the file it's running. In the REPL, `help(questkit)` shows your docstrings. That's what they're for.

Exercise 5 Biscuit's raffle

6 min · `ex5_raffle.py`

> **Raven from Captain Siv:** *"The lantern, the Wyrm watch and the marching order are all currently decided by the Quartermistress 'at random'. Everyone has noticed it's always the dwarves. Make it actually random. But the same random every time, so nobody can argue."*

GIVEN

```
import random

random.seed(1019)
names = ["Wren", "Bram", "Tamsin", "Osric", "Maren", "Elowen", "Dunstan", "Freya"]
```

(a) Print who carries the lantern with `random.choice`, then three different volunteers for the Wyrm watch with `random.sample`, joined with ", ".

(b) `random.shuffle` the list and print the first four as the marching order, joined with " -> ".

(c) Roll a die with `random.randint(1, 6)` for the length of Biscuit's flight.

EXPECTED OUTPUT

```
Carries the lantern: Wren
Wyrm watch volunteers: Elowen, Bram, Wren
Marching order: Wren -> Tamsin -> Dunstan -> Bram
Dice for Biscuit's flight: 4
```

Hint: The seed makes "random" repeatable. Keep the calls in exactly this order, or you'll get different (equally random) names. Remove the seed line and run twice to see the difference.

Exercise 6 Something you didn't write

8 min · `ex6_rich_table.py`

> **Decree from Queen Aldara:** *"Can the terminal output look less like a tax stone? Orrin says there's 'a package for that'. There's always a package for that."*

GIVEN

```
from rich.console import Console
from rich.table import Table

board = [("Q0001", "Goblins", "Thornwood", 260),
         ("Q0002", "Bandits", "Saltmarsh", 550),
         ("Q0004", "Bandits", "Northern Wilds", 670)]
```

(a) Make a `Table(title="The Notice Board")` with columns Quest, Monster, Region and Reward (right-justified). Add one row per quest, the reward as crowns with commas.

(b) Print it with `Console(width=60).print(table)`, then a second line with "3 quests" in bold green: `"[bold green]3 quests[/bold green], no Grimoire involved"`.

EXPECTED OUTPUT (THE BOX LINES MAY LOOK SLIGHTLY DIFFERENT IN YOUR TERMINAL)

```
              The Notice Board
┏━━━━━━━┳━━━━━━━━━┳━━━━━━━━━━━━━━┳━━━━━━━━┓
┃ Quest ┃ Monster ┃ Region       ┃ Reward ┃
┡━━━━━━━╇━━━━━━━━━╇━━━━━━━━━━━━━━╇━━━━━━━━┩
│ Q0001 │ Goblins │ Thornwood    │ 260 cr │
│ Q0002 │ Bandits │ Saltmarsh    │ 550 cr │
│ Q0004 │ Bandits │ Northern Wilds │ 670 cr │
└───────┴─────────┴──────────────┴────────┘
3 quests, no Grimoire involved
```

Hint: You've never seen `rich` before. That's the exercise: read the *Given*, guess from the names (`table.add_column(...)`, `table.add_row(...)`), and when stuck, search "rich table python". Reading other people's documentation is most of programming.

Exercise 7 The main guard STRETCH

8 min · `questkit.py` + `ex7_main_guard.py`

> **Raven from Orrin:** *"One more trick. A module should be quiet when imported and chatty when run on its own. There's a line for that. You've seen it before, in lesson 00, and nodded politely."*

GIVEN (ADD TO THE END OF `QUESTKIT.PY`)

```
if __name__ == "__main__":
    print("questkit self-test")
```

(a) Under that line, print a one-line check of each function, in the format shown below.

(b) Run `python my_work/questkit.py`: you get the self-test.

(c) Write `ex7_main_guard.py`: `from questkit import share, crowns`, then print the two lines below. Run it: no self-test appears.

EXPECTED OUTPUT OF `PYTHON MY_WORK/QUESTKIT.PY`

```
questkit self-test
share(550, 4)          -> 110.0
per_day(550, 7)        -> 78.57
fled_payout(2710)      -> 677.5
fled_payout(2710, 0.5) -> 1355.0
crowns(2708.333)       -> 2,708.33 cr
```

EXPECTED OUTPUT OF `PYTHON MY_WORK/EX7_MAIN_GUARD.PY`

```
Importing questkit printed nothing above this line.
One share of 1,800 cr among 5 heroes: 300.00 cr
```

Hint: `__name__` is `"__main__"` only in the file you ran directly. Inside an imported module it's the module's name, so the block is skipped.

Bug hunt Pages from the Grimoire, page 7

15 min · `bughunt/party_bug.py`

Dalvric's party planner for the guild's spring muster, written the week he discovered `import`. There are **three** bugs: one crashes on the very first line, one rounds the wrong way so one hero is left standing in the yard, and one prints something technically true but not what was asked.

EXPECTED OUTPUT ONCE FIXED

```
Heroes at the muster: 9
Average level: 8.33
Parties of 4 needed: 3
Days since Dalvric sailed west: 367653
```

Commit each fix separately, with a message explaining what was wrong.

More riddles. Five more torn pages wait in `bughunt/riddles/`, one bug each (the last one has two), getting harder as you go. Optional: for fast finishers, or a quiet evening.